



Universidade do Minho
Escola de Engenharia

Relatório do Projeto Final

Base de Dados Clínicas e de Gestão Hospitalar

Licenciatura em Engenharia Biomédica

Grupo 17:

A101461, Ana Cardoso

A100466, Beatriz Cunha

A101394, Carlota Brito

A101473, João Gil

Docentes:

Hugo Peixoto

José Machado



Resumo

Este relatório apresenta o trabalho realizado no âmbito da Unidade Curricular de Base de Dados Clínicas e de Gestão Hospitalar, como forma de consolidar os conteúdos abordados ao longo do semestre.

O trabalho mencionado consistiu na criação e manipulação de uma base de dados, sobre a qual se construiu um modelo conceptual, lógico e físico que permitisse uma interpretação simplificada dos dados associados. Para além disso, foi proposta a criação de um problema ao qual se pudessem apresentar soluções ao nível de arquitetura e povoamento e estabeleceram-se objetivos como o desenvolvimento de Querys, Funções, Procedimentos e Triggers que se apliquem ao caso de estudo.



Índice

1	Introdução	1
1.1	Contextualização	1
1.2	Caso de Estudo	1
2	Modelo Conceptual	2
2.1	Entidades	2
2.2	Atributos	3
3	Modelo Lógico	6
4	Modelo Físico	6
5	Documentação de Atributos	7
6	Povoamento	7
6.1	Povoamento da entidade “TELEFONE”	8
6.2	Povoamento da entidade “HOSPITAL”	8
6.3	Povoamento da entidade “CODIGO_POSTAL”	8
6.4	Povoamento da entidade “TRIAGEM”	8
6.5	Povoamento da entidade “UTENTE”	8
6.6	Povoamento da entidade “DIAGNOSTICO”	8
6.7	Povoamento da entidade “MEDICO”	8
6.8	Povoamento da entidade “CONSULTA”	9
6.9	Povoamento da entidade “BLOCO”	9
6.10	Povoamento da entidade “CIRURGIA”	9
7	Funções	9
7.1	Função Idade	9
7.2	Função Próxima Cirurgia	10
7.3	Função Número Total de Cirurgias por Médico	10
8	Procedimentos	11
8.1	Procedimento ‘ <i>inserir_medico</i> ’	11
8.2	Procedimento ‘ <i>listar_cirurgias_bloco</i> ’	12



9	<i>Triggers</i>	13
9.1	<i>Trigger “verificar_disponibilidade_bloco”</i>	13
9.2	<i>Tigger “atribuir_pulseira”</i>	14
10	Queries	15
10.1	Médicos com mais de 45 anos	15
10.2	Médicos que realizaram cirurgias de urgência nos últimos 6 meses	15
10.3	Pacientes com pulseira vermelha	16
10.4	Médicos que realizaram cirurgias mais que uma vez	17
10.5	Diagnóstico mais comum	17
10.6	Duração média das cirurgias	18
10.7	Próxima cirurgia	19
10.8	Próxima consulta	19
11	Conclusão	20
12	Anexos	21
12.1	Anexo A: Modelo Físico	21
12.2	Anexo B: Povoamento da entidade “TELEFONE”	26
12.3	Anexo C: Povoamento da entidade “HOSPITAL”	26
12.4	Anexo D: Povoamento da entidade “CODIGO_POSTAL”	27
12.5	Anexo E: Povoamento da entidade “TRIAGEM”	27
12.6	Anexo F: Povoamento da entidade “UTENTE”	28
12.7	Anexo G: Povoamento da entidade “DIAGNOSTICO”	28
12.8	Anexo H: Povoamento da entidade “MEDICO”	28
12.9	Anexo I: Povoamento da entidade “CONSULTA”	29
12.10	Anexo J: Povoamento da entidade “BLOCO”	29
12.11	Anexo K: Povoamento da entidade “CIRURGIA”	29



Lista de Figuras

1	Modelo Conceptual	2
2	Modelo Lógico	6
3	Tabela de documentação de atributos	7
4	Função idade	9
5	Função proxima_cirurgia	10
6	Testagem da função proxima_cirurgia	10
7	Função total_cirurgias_por_medico	10
8	Testagem da função total_cirurgias_por_medico	10
9	Procedimento 'inserir_medico'	11
10	(a) Resultado inserindo novos dados (b) Ativação do Procedimento	12
11	Procedimento 'listar_cirurgias_bloco'	12
12	(a) Resultado do Procedimento 'listar_cirurgias_bloco' (b) Ativação do Procedimento	13
13	Trigger 'verificar_disponibilidade_bloco'	13
14	Tentativa de adicionar uma cirurgia com um bloco ocupado	14
15	Resultado do Trigger 'verificar_disponibilidade_bloco'	14
16	Trigger 'atribuir_pulseira'	14
17	Testagem do Trigger 'atribuir_pulseira'	14
18	Query que retorna os médicos com mais de 45 anos	15
19	Resultado da query que retorna os médicos com mais de 45 anos	15
20	Query que retorna os médicos que realizaram cirurgias de urgência nos últimos 6 meses	15
21	(a) Query que retorna os médicos que realizaram cirurgias de urgência nos últimos 6 meses (b) Insert de um médico com cirurgias nos últimos 6 meses	16
22	Query que retorna os pacientes com pulseira vermelha	16
23	Resultado da query que retorna os pacientes com pulseira vermelha	17
24	Query que retorna os médicos que realizaram mais do que uma cirurgia	17
25	Resultado da query que retorna os médicos que realizaram mais do que uma cirurgia	17
26	Query que mostra o diagnóstico mais comum	17
27	Resultado da query que mostra o diagnóstico mais comum	18
28	Query que retorna a duração média das cirurgias	18
29	Resultado da query que retorna a duração média das cirurgias	18
30	Query que retorna informações sobre a próxima cirurgia	19
31	Resultado da query que retorna informações sobre a próxima cirurgia	19
32	Query que retorna informações sobre a próxima consulta	19



33	Resultado da query que retorna informações sobre a próxima consulta	20
34	Modelo Físico “TELEFONE”	21
35	Modelo Físico “CODIGO_POSTAL”	21
36	Modelo Físico “HOSPITAL”	21
37	Modelo Físico “ESPECIALIDADE”	21
38	Modelo Físico “MEDICO”	22
39	Modelo Físico “TRIAGEM”	22
40	Modelo Físico “UTENTE”	23
41	Modelo Físico “DIAGNOSTICO”	23
42	Modelo Físico “CONSULTA”	24
43	Modelo Físico “BLOCO”	24
44	Modelo Físico “CIRURGIA”	25
45	Povoamento da entidade “TELEFONE”	26
46	Povoamento da entidade “HOSPITAL”	26
47	Povoamento da entidade “CODIGO_POSTAL”	27
48	Povoamento da entidade “TRIAGEM”	27
49	Povoamento da entidade “UTENTE”	28
50	Povoamento da entidade “DIAGNOSTICO”	28
51	Povoamento da entidade “MEDICO”	28
52	Povoamento da entidade “CONSULTA”	29
53	Povoamento da entidade “BLOCO”	29
54	Povoamento da entidade “CIRURGIA”	29



1 Introdução

1.1 Contextualização

Na sua definição mais básica, uma base de dados é um conjunto de informações interrelacionadas. As bases de dados são utilizadas para armazenar e organizar dados para facilitar a gestão e o acesso aos mesmos. Os dados referem-se a qualquer informação capturada e armazenada sobre uma entidade, bem como os atributos dessa entidade. À medida que uma coleção de dados cresce e assume maior complexidade, torna-se mais difícil manter os dados organizados e acessíveis. Uma base de dados surge como solução a esse problema, facilitando a compreensão e interpretação dos dados em questão.

Os Sistemas de Base de Dados (SBD) são sistemas que nos permitem armazenar, recuperar e relacionar dados dentro do nossa base de dados. Neste caso, foi escolhida a plataforma MySQL, que constitui, atualmente, um dos sistemas mais utilizados nesta área. O MySQL é uma plataforma de gestão de dados, usada para gerir e manusear bases de dados relacionais, que utiliza a linguagem SQL como interface.

As bases de dados são essenciais na área da saúde, sendo fulcrais para o bom funcionamento das instituições desta área, suportando, não apenas a operação diária das unidades de saúde, mas também impulsionando avanços em cuidados médicos, eficiência administrativa e inovação científica.

1.2 Caso de Estudo

O nosso caso de estudo consistiu na elaboração de um sistema de informações para um hospital que permitisse a gestão das cirurgias realizadas no mesmo, de acordo com a sua urgência. O objetivo foi criar um modelo intuitivo, no qual fosse simples a obtenção de informações sobre médicos, pacientes, consultas e cirurgias, e no qual se pudesse manipular e atualizar a base de dados associada de forma prática.

2 Modelo Conceptual

Um modelo conceptual é um diagrama ou ferramenta gráfica que representa visualmente as relações entre conceitos e ideias. O modelo conceptual é construído através da análise de uma base de dados. Para tal, dividem-se os dados em Entidades, Atributos, chaves primárias e estabelecem-se relacionamentos entre eles. A Figura 1 é um diagrama Entidade-Relacionamento (ER) que foi elaborado utilizando o programa TerraER.

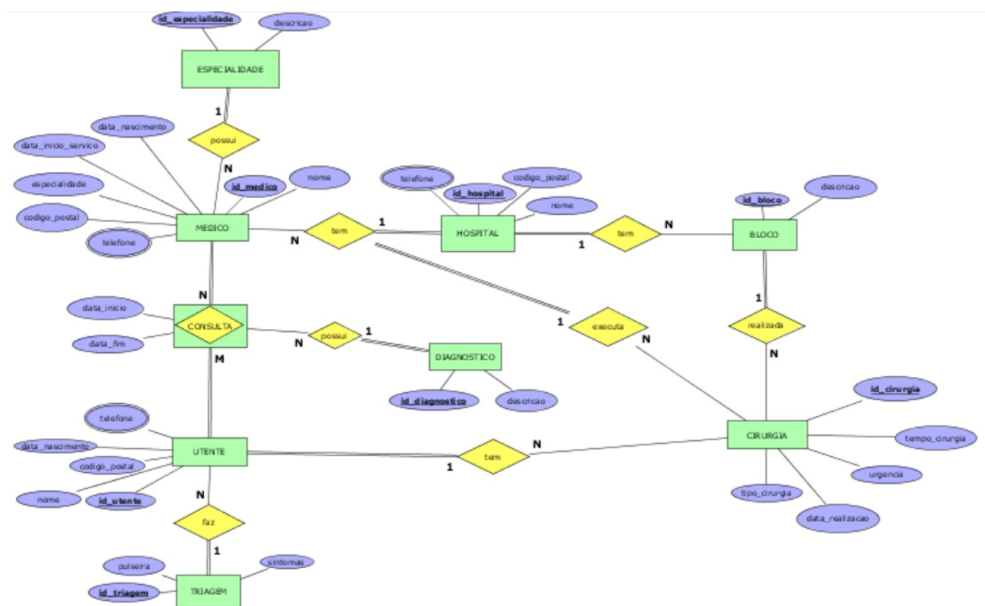


Figura 1: Modelo Conceptual

2.1 Entidades

Entidades são objetos ou conceitos representados no banco de dados.

Neste modelo, definiram-se as seguintes entidades:

- **MEDICO**: Armazena informações sobre os médicos;
- **ESPECIALIDADE**: Contém os diferentes tipos de especialidades médicas;
- **HOSPITAL**: Inclui detalhes sobre os hospitais;
- **CONSULTA**: Regista informações sobre consultas médicas;
- **DIAGNOSTICO**: Armazena descrições de diferentes diagnósticos médicos;
- **UTENTE**: Guarda os dados dos pacientes;
- **TRIAGEM**: Contém informações sobre a triagem pré cirurgica dos pacientes;



- CIRURGIA: Regista informações sobre cirurgias realizadas;
- BLOCO: Descreve os blocos cirúrgicos, ligados a um hospital específico;
- TELEFONE: Mantém um registo dos números de telefone dos médicos, pacientes e hospitais.

2.2 Atributos

Atributos são características ou propriedades que descrevem uma entidade. Cada atributo tem um tipo de dado associado, que define o tipo de valor que o atributo pode armazenar, como números inteiros, texto, datas, entre outros.

De seguida, apresentam-se os atributos associados a cada uma das entidades.

MEDICO:

- id_medico - Identificador único do médico
- nome - Nome do médico
- data_nascimento - Data de nascimento do médico
- data_inicio_servico - Data de início do serviço do médico no hospital
- especialidade - Identificador da especialidade do médico
- codigo_postal - Código postal do médico
- telefone - Telefone do médico
- hospital_id - Identificador do hospital onde o médico trabalha

HOSPITAL:

- id_hospital - Identificador único do hospital
- nome - Nome do hospital
- codigo_postal - Código postal do hospital
- telefone - Telefone do hospital

BLOCO

- id_bloco - Identificador único do bloco cirúrgico
- descricao - Descrição do bloco cirúrgico



- hospital_id - Identificador do hospital ao qual o bloco pertence

CIRURGIA:

- id_cirurgia - Identificador único da cirurgia
- data_realizacao - Data e hora de realização da cirurgia
- tempo_cirurgia - Duração da cirurgia
- tipo_cirurgia - Tipo de cirurgia realizada
- bloco_id - Identificador do bloco cirúrgico onde a cirurgia foi realizada
- utente_id - Identificador do paciente que passou pela cirurgia
- urgencia - Nível de urgência da cirurgia
- medico_id - Identificador do médico que realizou a cirurgia

CONSULTA:

- medico_id - Identificador do médico que realizou a consulta
- utente_id - Identificador do paciente que participou da consulta
- data_inicio - Data e hora de início da consulta
- data_fim - Data e hora de fim da consulta
diagnostico - Identificador do diagnóstico realizado na consulta

DIAGNOSTICO:

- id_diagnostico - Identificador único do diagnóstico
- descricao - Descrição do diagnóstico

UTENTE:

- id_utente - Identificador único do paciente
- nome - Nome do paciente
- codigo_postal - Código postal do paciente
- data_nascimento - Data de nascimento do paciente



- telefone - Telefone do paciente
- triagem_id - Identificador da triagem do paciente

ESPECIALIDADE:

- id_especialidade - Identificador único da especialidade
- descricao - Descrição da especialidade

TRIAGEM

- id_triagem - Identificador único da triagem
- pulseira - Cor da pulseira de triagem
- sintomas - Sintomas apresentados pelo paciente

TELEFONE

- telefone - Número de telefone

CODIGO_POSTAL

- n_codigo_postal - Número do código postal
- localidade - Localidade correspondente ao código postal

3 Modelo Lógico

Com um modelo de dados lógico, é possível visualizar processos operacionais ou transacionais em diagramas de relacionamento entre entidades. Os modelos de dados lógicos definem como os objetos de dados operam e se relacionam. Dessa forma, são projetados independentemente da base de dados real no qual serão implantados posteriormente.

Com base no modelo conceitual anterior, foi elaborado um modelo lógico em MySQL. Este modelo vai servir também ponto de partida para a criação do modelo físico.

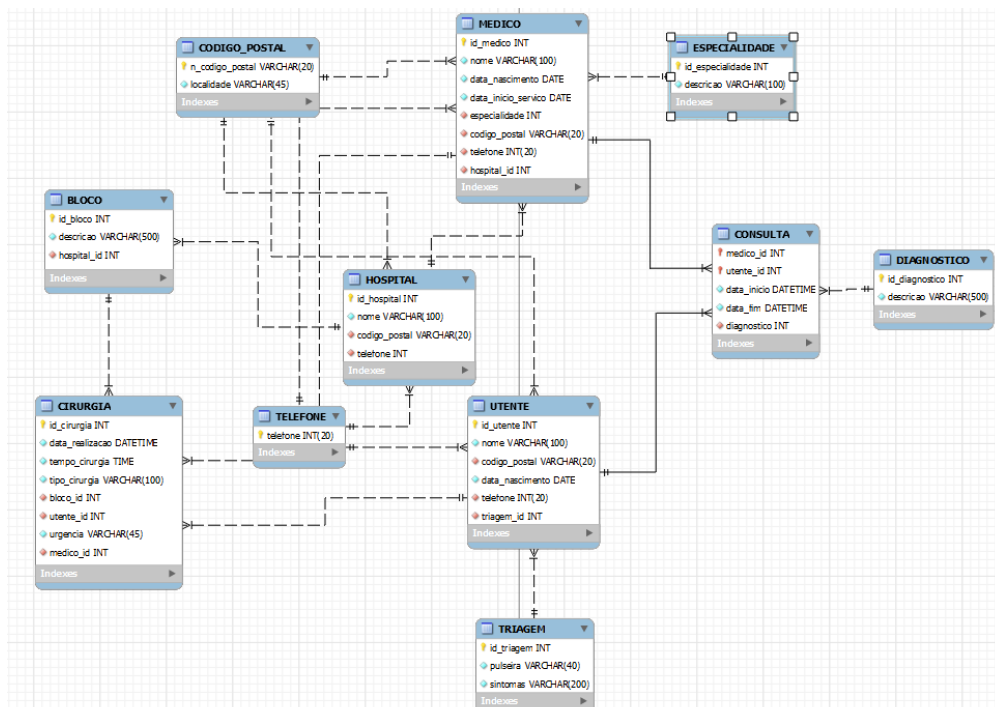


Figura 2: Modelo Lógico

4 Modelo Físico

Um modelo físico de dados em SQL é uma representação concreta de como os dados serão armazenados numa base de dados relacional. Este modelo inclui a definição das tabelas, colunas, tipos de dados, chaves primárias, chaves estrangeiras, índices e outras restrições necessárias para garantir a integridade e a eficiência da base de dados.

O modelo físico desenvolvido encontra-se no Anexo A.



5 Documentação de Atributos

Na tabela apresentada em baixo é possível observar uma classificação detalhada de cada atributo. Destacam-se as colunas de “Tipos de dados e tamanho”, que indica o tipo de dado de cada atributo (número inteiro (INT), data (DATE), caracteres variáveis (VARCHAR), entre outros. Para além disso, é de notar também as colunas chave primária e chave estrangeira. Uma chave primária é um atributo ou um conjunto de atributos que identifica unicamente cada tupla registo numa tabela (id_medico em MEDICO e id_bloco em BLOCO, por exemplo). As chaves estrangeiras são atributos que criam uma ligação entre duas tabelas, referenciando a chave primária de uma delas (por exemplo, hospital_id na tabela BLOCO: referencia id_hospital na tabela HOSPITAL). Estas chaves são importantes na estruturação eficiente de um base de dados relacional, como o modelo lógico apresentado, assegurando tanto a integridade dos dados, quanto a sua acessibilidade e manipulabilidade

Nome da Entidade	Atributos	Descrição dos atributos	Tipos de dados e tamanho	Nulo	Multi-valor	Chave primária	Chave estrangeira	Atributo composto
MEDICO	id_medico	Identificador do médico	Inteiro	Não	Não	Sim	Não	Não
	nome	Nome do médico	100 caracteres	Não	Não	Não	Não	Não
	data_nascimento	Data de nascimento	Data	Não	Não	Não	Não	Não
	data_inicio_servico	Data de início do serviço	Data	Não	Não	Não	Não	Não
	especialidade	Especialidade	60 caracteres	Não	Não	Não	Não	Não
	codigo_postal	Código Postal do médico	20 caracteres	Não	Não	Não	Sim	Sim
	telefone	Contacto do médico	20 algarismos	Não	Sim	Não	Sim	Não
	hospital	Nome do hospital	Inteiro	Não	Não	Não	Sim	Não
HOSPITAL	id_hospital	Identificador do hospital	Inteiro	Não	Não	Sim	Não	Não
	nome	Nome do hospital	100 caracteres	Não	Não	Não	Não	Não
	codigo_postal	Código Postal do hospital	20 caracteres	Não	Não	Não	Sim	Sim
	telefone	Contacto do hospital	Inteiro	Não	Sim	Não	Sim	Não
BLOCO	id_bloco	Identificador do bloco	Inteiro	Não	Não	Sim	Não	Não
	descricao	Descrição do bloco	500 caracteres	Não	Não	Não	Não	Não
CIRURGIA	hospital_id	Identificador do hospital	Inteiro	Não	Não	Não	Sim	Não
	id_cirurgia	Identificador de cirurgia	Inteiro	Não	Não	Sim	Não	Não
	data_realizacao	Data de realização da cirurgia	DataTempo	Não	Não	Não	Não	Não
	tempo_cirurgia	Duração da cirurgia	Tempo	Não	Não	Não	Não	Não
	tipo_cirurgia	Tipo de cirurgia	100 caracteres	Não	Não	Não	Não	Não
	bloco_id	Identificador do bloco	Inteiro	Não	Não	Não	Sim	Não
	utente_id	Identificador do utente	Inteiro	Não	Não	Não	Sim	Não
	urgencia	Nível de urgência	45 caracteres	Não	Não	Não	Não	Não
UTENTE	medico_id	Identificador do médico	Inteiro	Não	Não	Não	Sim	Não
	id_utente	Identificador do utente	Inteiro	Não	Não	Sim	Não	Não
	nome	Nome do utente	100 caracteres	Não	Não	Não	Não	Não
	codigo_postal	Código postal do utente	20 caracteres	Não	Não	Não	Sim	Sim
	data_nascimento	Data de nascimento do utente	Data	Não	Não	Não	Não	Não
TRIAGEM	telefone	Contacto do utente	20 algarismos	Não	Sim	Não	Sim	Não
	triagem_id	Identificador da triagem do utente	Inteiro	Não	Não	Não	Sim	Não
	id_triagem	Identificador da triagem	Inteiro	Não	Não	Sim	Não	Não
DIAGNOSTICO	pulseira	Cor da pulseira	40 caracteres	Não	Não	Não	Não	Não
	sintomas	Descrição dos sintomas	200 caracteres	Não	Não	Não	Não	Não
	id_diagnostico	Identificador do diagnóstico	Inteiro	Não	Não	Sim	Não	Não
CONSULTA	descricao	Descrição do diagnóstico	500 caracteres	Não	Não	Não	Não	Não
	medico_id	Identificador do médico	Inteiro	Não	Não	Sim	Sim	Não
	utente_id	Identificador do utente	Inteiro	Não	Não	Sim	Sim	Não
	data_inicio	Data e hora de início da consulta	DataTempo	Não	Não	Não	Não	Não
	data_fim	Data e hora de fim da consulta	DataTempo	Não	Não	Não	Não	Não
ESPECIALIDADE	diagnostico	Descrição do diagnóstico	Inteiro	Não	Não	Não	Sim	Não
	id_especialidade	Identificador de especialidade	Inteiro	Não	Não	Sim	Não	Não
	descricao	Descrição da especialidade	100 caracteres	Não	Não	Não	Não	Não

Figura 3: Tabela de documentação de atributos

6 Povoamento

De modo a criar a base de dados adequada ao problema proposto, procedeu-se ao povoamento, neste caso, feito de forma manual.



6.1 Povoamento da entidade “TELEFONE”

A entidade “TELEFONE” foi povoada utilizando números de telemóvel, para médicos e pacientes, e números fixos, para hospitais (Anexo B).

6.2 Povoamento da entidade “HOSPITAL”

Foram inseridos na base de dados 4 hospitais, em quatro cidades diferentes. Este conjunto de hospitais serve como exemplo para explicar as aplicações e funcionamento do trabalho desenvolvido (Anexo C).

6.3 Povoamento da entidade “CODIGO_POSTAL”

Ao povoar o “CODIGO_POSTAL” escolheram-se códigos postais e cidades correspondentes aos hospitais mencionados no povoamento anterior (Anexo D).

6.4 Povoamento da entidade “TRIAGEM”

No povoamento da “TRIAGEM”, introduziram-se casos para as três cores possíveis de pulseira, verde, amarela e vermelha, que refletem, por ordem crescente, o nível de gravidade do estado do paciente. Para além disso, para cada registo da triagem, adicionaram-se ainda os sintomas que levaram à atribuição de cada uma delas (Anexo E).

6.5 Povoamento da entidade “UTENTE”

Para povoar a entidade “UTENTE”, inseriram-se exemplos de utentes, com os respetivos atributos (id, nome, codigo_postal, data_nascimento, telefone e triagem_id) (Anexo F).

6.6 Povoamento da entidade “DIAGNOSTICO”

Para manter uma base de dados coerente, a entidade “DIAGNOSTICO” foi povoada com exemplos de diagnóstico correspondentes aos sintomas da “TRIAGEM” (Anexo G).

6.7 Povoamento da entidade “MEDICO”

O povoamento da entidade “MEDICO” realizou-se à semelhança dos anteriores, ou seja, através da criação de exemplos. Neste caso, a entidade “MEDICO” foi caracterizada pelo id_medico, nome, data_nascimento, data_inicio_servico, especialidade, codigo_postal, telefone e hospital_id (Anexo H).



6.8 Povoamento da entidade “CONSULTA”

No povoamento da “CONSULTA”, introduziram-se as consultas associadas aos exemplos referidos anteriormente, com o respetivo `medico_id`, `utente_id`, `data_inicio`, `data_fim` e `diagnostico` (Anexo I).

6.9 Povoamento da entidade “BLOCO”

Para povoar o “BLOCO”, criaram-se alguns blocos cirúrgicos, identificados por letras, com os respetivos `hospital_id`, que identificam o hospital em que cada um se situa (Anexo J).

6.10 Povoamento da entidade “CIRURGIA”

Para povoar a entidade “CIRURGIA”, inseriram-se exemplos de cirurgias relacionadas com os diagnósticos apresentados anteriormente. Os atributos associados a esta entidade (`id_cirurgia`, `data_realização`, `tempo_cirurgia`, `tipo_cirurgia`, `bloco_id`, `utente_id`, `urgencia` e `medico_id`) estabelecem a relação com as entidades anteriores (Anexo K).

7 Funções

Funções em SQL são utilizadas para realizar cálculos ou ações específicas, devolvendo um valor ou um conjunto de valores. Existem funções pré-definidas como AVG, COUNT e MIN/MAX mas também podem ser criadas pelo utilizador.

De seguida, serão apresentados exemplos de funções criadas no contexto do caso em estudo.

7.1 Função Idade

Foi criada a função `idade`, que permite determinar a idade a partir de uma data.

```
delimiter $$
create function idade (dta date)
returns int
begin
    return TIMESTAMPDIFF(YEAR,dta,CURDATE());
end $$
```

Figura 4: Função idade

A função está estabelecida de forma a receber um argumento `dta` do tipo `'DATE'` e a retornar um valor inteiro (`'RETURNS INT'`). Dentro do bloco `'BEGIN ... END'`, foi utilizado `'TIMESTAMPDIFF(YEAR, dta, CURDATE())'` para calcular a diferença em anos entre a data fornecida (`'dta'`) e a data atual (`'CURDATE()'`), que é o valor retornado no fim da função.



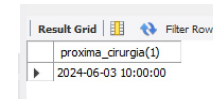
7.2 Função Próxima Cirurgia

A função '*proxima_cirurgia*' foi desenvolvida para encontrar a data e hora da próxima cirurgia agendada para um determinado bloco cirúrgico. Esta função permite a identificação rápida da próxima cirurgia agendada, facilitando a gestão dos blocos cirúrgicos.

```
DELIMITER $$
CREATE FUNCTION proxima_cirurgia(bloco INT)
RETURNS DATETIME
BEGIN
    DECLARE proxima DATETIME;

    SELECT MIN(data_realizacao) INTO proxima FROM CIRURGIA c, BLOCO b
    WHERE b.id_bloco=c.bloco_id AND c.bloco_id = bloco AND data_realizacao > NOW();

    RETURN proxima;
END$$
DELIMITER ;
```



proxima_cirurgia(1)
2024-06-03 10:00:00

Figura 5: Função proxima_cirurgia

Figura 6: Testagem da função
proxima_cirurgia

Esta função recebe um argumento '*bloco*' (identificador do bloco cirúrgico em questão) do tipo '*INT*' e retorna um valor do tipo '*DATETIME*'. Dentro da função, uma variável '*proxima*' é declarada para armazenar a data e hora da próxima cirurgia. A função executa uma consulta SQL que seleciona a menor data de realização de cirurgia, '*MIN(data_realizacao)*', de forma a garantir que a data obtida corresponde à próxima cirurgia. A parte da função '*data_realizacao > NOW()*' exige que a data retornada corresponda a uma cirurgia que ainda não aconteceu. '*WHERE b.id_bloco = c.bloco_id AND c.bloco_id = bloco*' garantem que a cirurgia ocorre no bloco específico e que o bloco é especificado como argumento da função. O resultado é atribuído à variável '*proxima*' e retornado pela função. Esta função é essencial para a gestão eficiente dos blocos cirúrgicos, permitindo a identificação rápida da próxima cirurgia agendada.

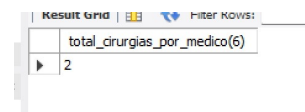
7.3 Função Número Total de Cirurgias por Médico

A função '*total_cirurgias_por_medico*' é projetada para calcular o número total de cirurgias realizadas por um certo médico. Esta função é útil para elaboração de relatórios e análises de desempenho, permitindo a avaliação da carga de trabalho e eficiência dos médicos.

```
DELIMITER $$
CREATE FUNCTION total_cirurgias_por_medico(medico INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE total INT;

    SELECT COUNT(*)
    INTO total
    FROM Trabalho_pratico.CIRURGIA
    WHERE medico_id = medico;

    RETURN total;
END$$
DELIMITER ;
```



total_cirurgias_por_medico(6)
2

Figura 7: Função total_cirurgias_por_medico

Figura 8: Testagem da função to-
tal_cirurgias_por_medico



8 Procedimentos

Um Procedimento em SQL são blocos de código armazenados na base de dados que podem ser executados repetidamente. Eles permitem a execução de operações complexas, como inserções, atualizações, e exclusões, além de lógica de controle. São utilizados para automação e padronização de tarefas no banco de dados.

8.1 Procedimento *'inserir_medico'*

O objetivo do procedimento *'inserir_medico'* é automatizar a inserção de registos na tabela MEDICO da base de dados *'Trabalho_prático'*. Isto garante que a operação de inserção seja realizada de maneira consistente e eficiente, evitando erros comuns que podem ocorrer ao inserir dados manualmente.

```
DELIMITER //
```

```
CREATE PROCEDURE inserir_medico (  
    IN p_id_medico INT,  
    IN p_nome VARCHAR(100),  
    IN p_data_nascimento DATE,  
    IN p_data_inicio_servico DATE,  
    IN p_especialidade INT,  
    IN p_codigo_postal VARCHAR(20),  
    IN p_telefone INT,  
    IN p_hospital_id INT)  
BEGIN  
    INSERT INTO Trabalho_prático.MEDICO (  
        id_medico, nome, data_nascimento, data_inicio_servico, especialidade, codigo_postal, telefone, hospital_id  
    ) VALUES (  
        p_id_medico, p_nome, p_data_nascimento, p_data_inicio_servico, p_especialidade, p_codigo_postal, p_telefone, p_hospital_id  
    );  
END //
```

```
DELIMITER ;
```

Figura 9: Procedimento *'inserir_medico'*

O procedimento é definido com o nome *'inserir_medico'* e aceita oito parâmetros de entrada: *'p_id_medico'*, *'p_nome'*, *'p_data_nascimento'*, *'p_data_inicio_servico'*, *'p_especialidade'*, *'p_codigo_postal'*, *'p_telefone'* e *'p_hospital_id'*. Esses parâmetros representam, respectivamente, o identificador único do médico, o seu nome, data de nascimento, data de início de serviço, especialidade, código postal, telefone e o identificador do hospital onde trabalha.

Dentro do corpo do procedimento, delimitado pelas palavras-chave BEGIN e END, é executada uma instrução INSERT INTO que insere os valores fornecidos nos parâmetros nas colunas correspondentes da tabela MEDICO. Isso garante que todos os campos necessários sejam preenchidos com os dados apropriados, mantendo a consistência e integridade dos registos no banco de dados.



	id_medico	nome	data_nascimento	data_inicio_servico	especialidade	codigo_postal	telefone	hospital_id
▶	15	João Gil	1990-01-06	2015-06-19	1	4700-001	961228720	4
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

(a)

```
CALL inserir_medico(15,'João Gil', '1990-01-06', '2015-06-19', 1,'4700-001', 961228720, 4);
```

(b)

Figura 10: (a) Resultado inserindo novos dados (b) Ativação do Procedimento

8.2 Procedimento 'listar_cirurgias_bloco'

O procedimento 'listar_cirurgias_bloco' foi criado para facilitar a consulta e listagem de cirurgias realizadas num determinado bloco cirúrgico dentro de um intervalo de datas específico.

```
DELIMITER $$  
  
CREATE PROCEDURE listar_cirurgias_bloco(IN p_bloco_id INT, IN p_data_inicio DATETIME, IN p_data_fim DATETIME)  
BEGIN  
    SELECT c.id_cirurgia, c.data_realizacao, c.tempo_cirurgia, c.tipo_cirurgia, c.utente_id, u.nome AS nome_utente, c.urgencia, c.data_realizacao FROM CIRURGIA c  
    JOIN  
        UTENTE u ON c.utente_id = u.id_utente  
    WHERE  
        c.bloco_id = p_bloco_id  
        AND c.data_realizacao BETWEEN p_data_inicio AND p_data_fim  
    ORDER BY  
        c.data_realizacao;  
END$$  
  
DELIMITER ;
```

Figura 11: Procedimento 'listar_cirurgias_bloco'

O procedimento definido aceita três parâmetros de entrada: 'p_bloco_id', 'p_data_inicio' e 'p_data_fim'. Estes parâmetros representam, respectivamente, o identificador do bloco cirúrgico, a data e hora de início do intervalo e a data e hora de fim do intervalo.

Dentro do corpo do procedimento, é executada uma instrução SELECT que consulta a tabela CIRURGIA, trazendo dados específicos sobre as cirurgias realizadas no bloco cirúrgico especificado (p_bloco_id) e que ocorreram entre as datas e horários fornecidos (p_data_inicio e p_data_fim). A consulta retorna as seguintes colunas da tabela CIRURGIA: id_cirurgia, data_realizacao, tempo_cirurgia, tipo_cirurgia, utente_id, urgencia e data_realizacao. Além disso, a consulta junta a tabela CIRURGIA com a tabela UTENTE para obter o nome do paciente (nome_utente), associando a utente_id da tabela CIRURGIA com a id_utente da tabela UTENTE.

A cláusula WHERE garante que apenas as cirurgias do bloco cirúrgico especificado e realizadas dentro do intervalo de datas fornecido sejam selecionadas. Os resultados são ordenados pela coluna data_realizacao para apresentar as cirurgias em ordem cronológica.



	id_cirurgia	data_realizacao	tempo_cirurgia	tipo_cirurgia	utente_id	nome_utente	urgencia	data_realizacao
▶	1	2024-06-01 08:00:00	02:00:00	Cardiologia	2	Ana Rita	urgente	2024-06-01 08:00:00

(a)

```
CALL listar_cirurgias_bloco(2, '2024-06-01 00:00:00', '2024-06-02 23:59:59');
```

(b)

Figura 12: (a) Resultado do Procedimento *'listar_cirurgias_bloco'* (b) Ativação do Procedimento

Com o procedimento criado, ele pode ser invocado posteriormente com uma instrução `CALL`, onde os valores dos parâmetros são fornecidos conforme necessário. Isso simplifica o processo de consulta de cirurgias, tornando-o mais eficiente e organizado.

9 Triggers

Um "*trigger*" em sistemas de gestão de bases de dados (SGBDs) é um procedimento automático que é executado em resposta a determinados eventos numa tabela. Geralmente, as ações que disparam os triggers são alterações nas tabelas por meio de operações de inserção, exclusão e atualização de dados (*insert*, *delete* e *update*). Triggers são usados para garantir a integridade dos dados e automatizar tarefas como atualizações, entre outras funções.

9.1 Trigger "*verificar_disponibilidade_bloco*"

Criou-se um *trigger* com o objetivo de impedir a inserção de uma nova cirurgia se o bloco cirúrgico já estiver ocupado no período solicitado, garantindo assim que não haja sobreposição de horários de cirurgias no mesmo bloco.

```
DELIMITER $$

CREATE TRIGGER verificar_disponibilidade_bloco
BEFORE INSERT ON Trabalho_prático.CIRURGIA
FOR EACH ROW
BEGIN
    DECLARE bloco_ocupado INT;
    DECLARE fim_nova_cirurgia DATETIME;

    SET fim_nova_cirurgia = DATE_ADD(NEW.data_realizacao, INTERVAL TIME_TO_SEC(TIMEDIFF(NEW.tempo_cirurgia, '00:00:00')) SECOND);

    SELECT COUNT(*)
    INTO bloco_ocupado
    FROM Trabalho_prático.CIRURGIA
    WHERE bloco_id = NEW.bloco_id
    AND (
        (NEW.data_realizacao BETWEEN data_realizacao AND DATE_ADD(data_realizacao, INTERVAL TIME_TO_SEC(TIMEDIFF(tempo_cirurgia, '00:00:00')) SECOND)) OR
        (fim_nova_cirurgia BETWEEN data_realizacao AND DATE_ADD(data_realizacao, INTERVAL TIME_TO_SEC(TIMEDIFF(tempo_cirurgia, '00:00:00')) SECOND)) OR
        (data_realizacao BETWEEN NEW.data_realizacao AND fim_nova_cirurgia)
    );

    IF bloco_ocupado > 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Bloco cirúrgico não está disponível para esta data e hora.';
    END IF;
END$$

DELIMITER ;
```

Figura 13: Trigger *'verificar_disponibilidade_bloco'*

O *trigger* *'verificar_disponibilidade_bloco'* é ativado antes de qualquer inserção na tabela. Pri-

meiramente, são declaradas duas variáveis: ‘*bloco_ocupado*’, para armazenar a contagem de conflitos, e ‘*fim_nova_cirurgia*’, para calcular a data e hora de término da nova cirurgia. O término da nova cirurgia é calculado somando a duração da cirurgia (‘*NEW.tempo_cirurgia*’) à data de início (‘*NEW.data_realizacao*’). De seguida, é realizada uma consulta que conta quantas cirurgias já agendadas no mesmo bloco (‘*NEW.bloco_id*’) têm horários conflitantes com a nova cirurgia. A verificação de conflito considera três condições: se a nova cirurgia começa durante uma cirurgia já agendada, se a nova cirurgia termina durante uma cirurgia já agendada, ou se a cirurgia já agendada começa durante a nova cirurgia. Se a contagem de conflitos (‘*bloco_ocupado*’) for maior que zero, um erro é gerado com uma mensagem que informa que o bloco cirúrgico não está disponível para a data e hora especificadas.

```
INSERT INTO Trabalho_prático.CIRURGIA (id_cirurgia, data_realizacao, tempo_cirurgia, tipo_cirurgia, bloco_id, utente_id, urgencia, medico_id) VALUES
(13, '2024-06-01 09:00:00', '05:00:00', 'Cardiologia', 2, 2, 'muito urgente', 6);
```

Figura 14: Tentativa de adicionar uma cirurgia com um bloco ocupado

```
104 01:55:24 INSERT INTO Trabalho_prático.CIRURGIA (id_cirurgia, data_realizacao, tempo_cirurgia, tipo_cirurgia, bloco_id, utente_id, urgencia, medico_id) VALUES
(13, '2024-06-01 09:00:00', '05:00:00', 'Cardiologia', 2, 2, 'muito urgente', 6); Error Code: 1644. Bloco cirúrgico não está disponível para esta data e hora.
```

Figura 15: Resultado do *Trigger* ‘*verificar_disponibilidade_bloco*’

9.2 *Tigger* “*atribuir_pulseira*”

O trigger ‘*atribuir_pulseira*’ tem como objetivo a atribuição de uma cor de pulseira, baseada no nível de urgência de uma cirurgia, garantindo assim a correta categorização da prioridade da mesma.

```
DELIMITER $$

--CREATE TRIGGER atribuir_pulseira
BEFORE INSERT ON Trabalho_prático.CIRURGIA
FOR EACH ROW
BEGIN
    DECLARE pulseira VARCHAR(40);
    IF NEW.urgencia = 'pouco urgente' THEN
        SET pulseira = 'Verde';
    ELSEIF NEW.urgencia = 'urgente' THEN
        SET pulseira = 'Amarelo';
    ELSEIF NEW.urgencia = 'muito urgente' THEN
        SET pulseira = 'Vermelha';
    ELSE
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insira a prioridade de forma correta';
    END IF;

    UPDATE Trabalho_prático.TRIAGEM t
    JOIN Trabalho_prático.UTENTE u ON NEW.utente_id = u.id_utente
    SET t.pulseira = pulseira
    WHERE u.triagem_id = t.id_triagem;
END$$

DELIMITER ;
```

	nome	pulseira
▶	Ana Rita	Vermelha

Figura 16: *Trigger* ‘*atribuir_pulseira*’

Figura 17: Testagem do *Trigger* ‘*atribuir_pulseira*’

O trigger ‘*atribuir_pulseira*’ é executado antes da inserção de uma nova entrada na tabela ‘*Trabalho_prático.CIRURGIA*’ para determinar a cor da pulseira do paciente com base na urgência da

cirurgia. Este *trigger* declara a variável ‘*pulseira*’ e atribui uma cor específica à mesma (‘*Verde*’ para ‘*pouco urgente*’, ‘*Amarelo*’ para ‘*urgente*’, e ‘*Vermelha*’ para ‘*muito urgente*’). Se o valor da urgência não for reconhecido, o *trigger* gera um erro com a mensagem ‘*Insira a prioridade de forma correta*’. Caso contrário, a cor da pulseira é então atualizada na tabela ‘*Trabalho_prático.TRIAGEM*’, associando a triagem do paciente ao novo valor de urgência.

10 Queries

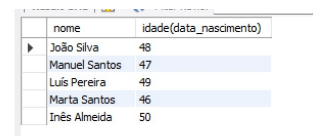
Uma Query constitui uma pergunta, consulta ou dúvida. É uma solicitação de informações feita à base de dados, que retorna uma tabela ou um conjunto de tabelas, figuras ou gráficos. As queries são ferramentas essenciais para a manipulação e gestão eficiente de dados, possibilitando que os usuários extraiam informações úteis de forma simples.

10.1 Médicos com mais de 45 anos

A finalidade desta query é listar os nomes e idades dos médicos que têm mais de 45 anos.

```
SELECT nome, idade(data_nascimento) FROM MEDICO
WHERE YEAR(CURDATE()) - YEAR(data_nascimento) > '45';
```

Figura 18: Query que retorna os médicos com mais de 45 anos



nome	idade(data_nascimento)
João Silva	48
Manuel Santos	47
Luís Pereira	49
Marta Santos	46
Inês Almeida	50

Figura 19: Resultado da query que retorna os médicos com mais de 45 anos

O código cumpre esse objetivo utilizando a função ‘*SELECT*’ para extrair as colunas ‘*nome*’ e ‘*idade(data_nascimento)*’ da tabela ‘*MEDICO*’. A função ‘*idade(data_nascimento)*’, definida anteriormente, calcula a idade a partir da data de nascimento. O filtro ‘*WHERE YEAR(CURDATE()) - YEAR(data_nascimento) > 45*’ compara o ano atual (obtido por ‘*YEAR(CURDATE())*’) com o ano de nascimento de cada médico (‘*YEAR(data_nascimento)*’) e seleciona apenas aqueles médicos cuja idade é maior que 45 anos, retornando esse conjunto como resultado.

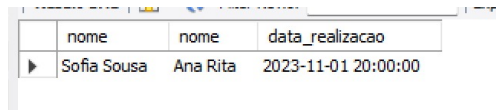
10.2 Médicos que realizaram cirurgias de urgência nos últimos 6 meses

Esta query seleciona os nomes dos médicos e dos pacientes, além das datas de realização das cirurgias que foram classificadas como “muito urgentes” e ocorreram nos últimos seis meses.

```
SELECT m.nome, u.nome, c.data_realizacao FROM MEDICO m, CIRURGIA c, UTENTE u
WHERE m.id_medico = c.medico_id AND c.utente_id = u.id_utente AND c.urgencia = 'muito urgente' AND c.data_realizacao <= CURDATE() - INTERVAL 6 MONTH;
```

Figura 20: Query que retorna os médicos que realizaram cirurgias de urgência nos últimos 6 meses

É utilizada a cláusula *'SELECT'* para extrair as colunas *'m.nome'* (nome do médico), *'u.nome'* (nome do paciente) e *'c.data_realizacao'* (data da realização da cirurgia) das tabelas *'MEDICO'*, *'CIRURGIA'* e *'UTENTE'*, respetivamente. A junção das tabelas é feita com base nas chaves estrangeiras *'m.id_medico = c.medico_id'* e *'c.utente_id = u.id_utente'*, garantindo que apenas os registos correspondentes sejam combinados. A cláusula *'WHERE'* aplica os filtros: *'c.urgencia = 'muito urgente''* para seleccionar apenas as cirurgias classificadas como muito urgentes e *'c.data_realizacao <= CURDATE() - INTERVAL 6 MONTH'* para garantir que a data de realização da cirurgia seja nos últimos seis meses.



nome	nome	data_realizacao
Sofia Sousa	Ana Rita	2023-11-01 20:00:00

(a)

```
INSERT INTO Trabalho_prático.CIRURGIA (id_cirurgia, data_realizacao, tempo_cirurgia, tipo_cirurgia, bloco_id, utente_id, urgencia, medico_id) VALUES  
(23, '2023-11-01 20:00:00', '02:00:00', 'Cardiologia', 2, 2, 'muito urgente', 6);
```

(b)

Figura 21: (a) Query que retorna os médicos que realizaram cirurgias de urgência nos últimos 6 meses (b) Insert de um médico com cirurgias nos últimos 6 meses

10.3 Pacientes com pulseira vermelha

Este query identifica e retorna os IDs e nomes de todos os utentes que foram classificados como *'muito urgente'* (indicados pela pulseira vermelha).

```
SELECT id_utente, nome FROM UTENTE, TRIAGEM  
WHERE TRIAGEM.pulseira = "Vermelha" AND UTENTE.triagem_id = TRIAGEM.id_triagem;
```

Figura 22: Query que retorna os pacientes com pulseira vermelha

Esta query SQL selecciona informações de duas tabelas diferentes, *'UTENTE'* e *'TRIAGEM'*. Especificamente, vai seleccionar os campos *'id_utente'* e *'nome'* da tabela *'UTENTE'* para todos os registos onde a pulseira (indicador de prioridade) na tabela *'TRIAGEM'* é *'vermelha'*. A condição adicional *'UTENTE.triagem_id = TRIAGEM.id_triagem'* estabelece a relação entre as duas tabelas, garantindo que cada utente seleccionado esteja associado corretamente a uma triagem com a pulseira vermelha.

id_utente	nome
2	Ana Rita
6	Maria Ferreira
9	Pedro Sousa
7	Luís Oliveira
12	Beatriz Cunha

Figura 23: Resultado da query que retorna os pacientes com pulseira vermelha

10.4 Médicos que realizaram cirurgias mais que uma vez

Esta query retorna os nomes dos médicos que realizaram mais de uma cirurgia e a quantidade dessas cirurgias.

```
SELECT m.nome, COUNT(c.id_cirurgia) FROM MEDICO m, CIRURGIA c
WHERE m.id_medico = c.medico_id
GROUP BY m.id_medico, m.nome
HAVING COUNT(c.id_cirurgia) > 1;
```

nome	COUNT(c.id_cirurgia)
Sofia Sousa	3

Figura 24: Query que retorna os médicos que realizaram mais do que uma cirurgia

Figura 25: Resultado da query que retorna os médicos que realizaram mais do que uma cirurgia

Esta query SQL seleciona os nomes dos médicos (*m.nome*) e o número de cirurgias realizadas por cada um (*COUNT(c.id_cirurgia)*) a partir das tabelas *MEDICO* e *CIRURGIA*. A condição *m.id_medico = c.medico_id* garante que apenas as cirurgias realizadas por cada médico específico sejam contadas, estabelecendo a relação entre as duas tabelas. O agrupamento (*GROUP BY m.id_medico, m.nome*) é feito pelos id e nomes dos médicos, permitindo que o número de cirurgias seja contado para cada médico individualmente. A cláusula *HAVING COUNT(c.id_cirurgia) > 1* filtra os resultados para incluir apenas médicos que realizaram mais do que uma cirurgia.

10.5 Diagnóstico mais comum

Esta query SQL é utilizada para gerar um relatório que mostra o diagnóstico mais comum e a quantidade de vezes que este foi registado em consultas médicas.

```
SELECT d.descricao, COUNT(co.diagnostico) FROM DIAGNOSTICO d, CONSULTA co
WHERE d.id_diagnostico = co.diagnostico
GROUP BY d.id_diagnostico, d.descricao
ORDER BY COUNT(co.diagnostico) DESC;
```

Figura 26: Query que mostra o diagnóstico mais comum

Primeiramente, seleciona a descrição do diagnóstico (*d.descricao*) e a contagem de ocorrências de cada diagnóstico (*COUNT(co.diagnostico)*) a partir das tabelas *DIAGNOSTICO* (abreviada como *d*) e *CONSULTA* (abreviada como *co*). A condição especificada no *WHERE* (*d.id_diagnostico = co.diagnostico*) relaciona as duas tabelas, garantindo que somente os registos onde o diagnóstico da consulta corresponde a um diagnóstico listado sejam considerados. A cláusula

‘*GROUP BY*’ agrupa os resultados pelo identificador do diagnóstico (‘*d.id_diagnostico*’) e a sua descrição (‘*d.descricao*’), permitindo que a contagem seja calculada para cada grupo. Finalmente, os resultados são ordenados por ordem decrescente pela contagem de diagnósticos (‘*ORDER BY COUNT(co.diagnostico) DESC*’), exibindo primeiro os diagnósticos mais frequentes.

	descricao	COUNT(co.diagnostico)
▶	AVC	2
	Sangue nos pulmões	2
	Dermatite atópica	2
	Fémur fraturado	2
	Gripe	1
	Osteoartrite	1
	Angina	1
	Urticária solar	1

Figura 27: Resultado da query que mostra o diagnóstico mais comum

10.6 Duração média das cirurgias

Esta query SQL é utilizada para calcular o tempo médio das cirurgias (em minutos) realizadas em diferentes blocos cirúrgicos de diversos hospitais, organizando os resultados por bloco. A query em questão retorna o nome do hospital, o id do bloco e a duração média das cirurgias

```
SELECT h.nome, b.id_bloco, AVG(TIME_TO_SEC(c.tempo_cirurgia)) / 60 FROM HOSPITAL h, BLOCO b, CIRURGIA c
WHERE h.id_hospital = b.hospital_id AND b.id_bloco = c.bloco_id
GROUP BY h.nome, b.id_bloco
ORDER BY id_bloco;
```

Figura 28: Query que retorna a duração média das cirurgias

Inicialmente, seleciona o nome do hospital (*h.nome*), o identificador do bloco cirúrgico (*b.id_bloco*) e a média do tempo de cirurgia convertido de segundos para minutos ($AVG(TIME_TO_SEC(c.tempo_cirurgia)) / 60$). As tabelas envolvidas são *HOSPITAL* (abreviada como *h*), *BLOCO* (abreviada como *b*) e *CIRURGIA* (abreviada como *c*). A condição no *WHERE* ($h.id_hospital = b.hospital_id$ AND $b.id_bloco = c.bloco_id$) estabelece as relações entre as três tabelas: cada bloco está associado a um hospital e cada cirurgia está associada a um bloco. A cláusula *GROUP BY* agrupa os resultados pelo nome do hospital (*h.nome*) e pelo identificador do bloco (*b.id_bloco*), permitindo calcular a média do tempo das cirurgias para cada grupo. Finalmente, os resultados são ordenados pelo identificador do bloco (*ORDER BY b.id_bloco*).

	nome	id_bloco	AVG(TIME_TO_SEC(c.tempo_cirurgia)) / 60
▶	Hospital de Santa Maria	1	420.00000000
	Hospital de São João	2	130.00000000
	Centro Hospitalar de Coimbra	3	300.00000000
	Hospital de Braga	4	240.00000000
	Hospital de Santa Maria	5	360.00000000
	Hospital de São João	6	120.00000000
	Centro Hospitalar de Coimbra	7	180.00000000
	Hospital de Braga	8	180.00000000

Figura 29: Resultado da query que retorna a duração média das cirurgias



10.7 Próxima cirurgia

A query SQL apresentada é usada para selecionar o nome dos médicos, a descrição dos blocos cirúrgicos e a data da próxima cirurgia em cada bloco.

```
SELECT m.nome, b.descricao, proxima_cirurgia(c.bloco_id) FROM MEDICO m, CIRURGIA c , BLOCO b
WHERE proxima_cirurgia(c.bloco_id) = c.data_realizacao AND c.medico_id = m.id_medico AND c.bloco_id = b.id_bloco;
```

Figura 30: Query que retorna informações sobre a próxima cirurgia

Como tal, consulta as tabelas *MEDICO*, *CIRURGIA* e *BLOCO*, relacionando-as pelos identificadores dos médicos e dos blocos. A seleção das colunas inclui o nome do médico (*m.nome*), a descrição do bloco cirúrgico (*b.descricao*) e o resultado da função *proxima_cirurgia*, aplicada ao identificador do bloco cirúrgico (*proxima_cirurgia(c.bloco_id)*). A condição *WHERE* filtra os resultados para incluir apenas as cirurgias cuja data coincide com a data retornada pela função *proxima_cirurgia*. As tabelas são relacionadas usando *JOIN*: *CIRURGIA* é relacionada com *MEDICO* através do identificador do médico (*c.medico_id = m.id_medico*), e a *BLOCO* através do identificador do bloco (*c.bloco_id = b.id_bloco*).

nome	descricao	proxima_cirurgia(c.bloco_id)
Sofia Sousa	Bloco Cirúrgico B	2024-06-01 08:00:00
Pedro Costa	Bloco Cirúrgico C	2024-06-02 09:00:00
João Silva	Bloco Cirúrgico A	2024-06-03 10:00:00
Maria Fernandes	Bloco Cirúrgico F	2024-06-04 11:00:00
Luís Pereira	Bloco Cirúrgico G	2024-06-05 12:00:00
Carolina Martins	Bloco Cirúrgico D	2024-06-06 13:00:00
Manuel Santos	Bloco Cirúrgico E	2024-06-07 14:00:00
Ana Oliveira	Bloco Cirúrgico H	2024-06-10 17:00:00

Figura 31: Resultado da query que retorna informações sobre a próxima cirurgia

10.8 Próxima consulta

Esta query seleciona o nome do utente e médico que vão ter a próxima consulta, em que hospital se vai realizar e a data de início das consulta.

```
SELECT c.data_inicio, m.nome AS nome_medico, u.nome AS nome_utente, h.nome AS nome_hospital
FROM CONSULTA c
JOIN UTENTE u ON c.utente_id = u.id_utente
JOIN MEDICO m ON c.medico_id = m.id_medico
JOIN HOSPITAL h ON m.hospital_id = h.id_hospital
ORDER BY c.data_inicio;
```

Figura 32: Query que retorna informações sobre a próxima consulta

Para isso, consulta as tabelas '*CONSULTA*', '*UTENTE*', '*MEDICO*' e '*HOSPITAL*', relacionando-as através de seus respectivos identificadores. A junção das tabelas é realizada da seguinte forma: '*CONSULTA*' é relacionada a '*UTENTE*' pelo identificador do utente ('*c.utente_id = u.id_utente*'), a '*MEDICO*' pelo identificador do médico ('*c.medico_id = m.id_medico*') e a '*HOSPITAL*' pelo



identificador do hospital (*'m.hospital_id = h.id_hospital'*). Seleccionam-se então as colunas *'c.data_inicio'* para a data de início da consulta, *'m.nome'* como *'nome_medico'* para o nome do médico, *'u.nome'* como *'nome_utente'* para o nome do utente, e *'h.nome'* como *'nome_hospital'* para o nome do hospital. Finalmente, os resultados são ordenados pela data de início das consultas (*'c.data_inicio'*) em ordem crescente.

	data_inicio	nome_medico	nome_utente	nome_hospital
▶	2024-05-01 10:00:00	Ricardo Ferreira	Carlos Mendes	Hospital de Santa Maria
	2024-05-02 11:00:00	Marta Santos	Ana Rita	Hospital de São João
	2024-05-03 14:00:00	Miguel Gonçalves	Joaquim Sousa	Centro Hospitalar de Coimbra
	2024-05-04 16:00:00	Inês Almeida	Maria João	Hospital de Braga
	2024-05-05 09:00:00	Ricardo Ferreira	João Silva	Hospital de Santa Maria
	2024-05-06 10:30:00	Marta Santos	Maria Ferreira	Hospital de São João
	2024-05-07 12:00:00	Miguel Gonçalves	Luís Oliveira	Centro Hospitalar de Coimbra
	2024-05-08 13:00:00	Inês Almeida	Ana Costa	Hospital de Braga
	2024-05-09 14:30:00	Ricardo Ferreira	Pedro Sousa	Hospital de Santa Maria
	2024-05-10 15:30:00	Marta Santos	Sofia Martins	Hospital de São João
	2024-05-11 16:30:00	Miguel Gonçalves	Tiago Rodrigues	Centro Hospitalar de Coimbra
	2024-05-12 17:00:00	Inês Almeida	Beatriz Cunha	Hospital de Braga

Figura 33: Resultado da query que retorna informações sobre a próxima consulta

11 Conclusão

Em conclusão, este trabalho apresentou o desenvolvimento e implementação de modelos, utilizando a plataforma MySQL, para a gestão de bases de dados relacionais.

O sistema desenvolvido demonstrou ser uma solução eficaz para a gestão das cirurgias realizadas no hospital, permitindo um fácil acesso e manipulação de dados sobre médicos, pacientes, consultas e procedimentos cirúrgicos.

Ao longo da realização deste projeto, foi perceptível a importância das bases de dados na organização e acessibilidade a informações, nomeadamente na área da saúde, onde a eficiência e a precisão são cruciais.

Assim, podemos concluir que este projeto se revelou extremamente útil na consolidação da matéria abordada na UC, aplicando-a num contexto mais prático e adequado ao dia-a-dia. Além disso, permitiu-nos compreender a utilidade, inter-relação e importância das ferramentas adquiridas na UC.



12 Anexos

12.1 Anexo A: Modelo Físico

```
CREATE TABLE IF NOT EXISTS `Trabalho_prático`.`TELEFONE` (  
  `telefone` INT(20) NOT NULL,  
  PRIMARY KEY (`telefone`))  
ENGINE = InnoDB;
```

Figura 34: Modelo Físico “TELEFONE”

```
CREATE TABLE IF NOT EXISTS `Trabalho_prático`.`CODIGO_POSTAL` (  
  `n_codigo_postal` VARCHAR(20) NOT NULL,  
  `localidade` VARCHAR(45) NOT NULL,  
  PRIMARY KEY (`n_codigo_postal`))  
ENGINE = InnoDB;
```

Figura 35: Modelo Físico “CODIGO_POSTAL”

```
CREATE TABLE IF NOT EXISTS `Trabalho_prático`.`HOSPITAL` (  
  `id_hospital` INT NOT NULL,  
  `nome` VARCHAR(100) NOT NULL,  
  `codigo_postal` VARCHAR(20) NOT NULL,  
  `telefone` INT NOT NULL,  
  PRIMARY KEY (`id_hospital`),  
  INDEX `fk1_telefone_hospital_idx` (`telefone` ASC) VISIBLE,  
  INDEX `fk2_cp_hospital_idx` (`codigo_postal` ASC) VISIBLE,  
  CONSTRAINT `fk1_telefone_hospital`  
    FOREIGN KEY (`telefone`)  
    REFERENCES `Trabalho_prático`.`TELEFONE` (`telefone`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION,  
  CONSTRAINT `fk2_cp_hospital`  
    FOREIGN KEY (`codigo_postal`)  
    REFERENCES `Trabalho_prático`.`CODIGO_POSTAL` (`n_codigo_postal`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

Figura 36: Modelo Físico “HOSPITAL”

```
CREATE TABLE IF NOT EXISTS `Trabalho_prático`.`ESPECIALIDADE` (  
  `id_especialidade` INT NOT NULL,  
  `descricao` VARCHAR(100) NOT NULL,  
  PRIMARY KEY (`id_especialidade`))  
ENGINE = InnoDB;
```

Figura 37: Modelo Físico “ESPECIALIDADE”

```
CREATE TABLE IF NOT EXISTS `Trabalho_prático`.`MEDICO` (  
  `id_medico` INT NOT NULL,  
  `nome` VARCHAR(100) NOT NULL,  
  `data_nascimento` DATE NOT NULL,  
  `data_inicio_servico` DATE NOT NULL,  
  `especialidade` INT NOT NULL,  
  `codigo_postal` VARCHAR(20) NOT NULL,  
  `telefone` INT(20) NOT NULL,  
  `hospital_id` INT NOT NULL,  
  PRIMARY KEY (`id_medico`),  
  INDEX `fk1_medico_telefone_idx` (`telefone` ASC) VISIBLE,  
  INDEX `fk2_hospital_medico_idx` (`hospital_id` ASC) VISIBLE,  
  INDEX `fk3_cp_medico_idx` (`codigo_postal` ASC) VISIBLE,  
  INDEX `fk5_especialidade_medico_idx` (`especialidade` ASC) VISIBLE,  
  CONSTRAINT `fk1_telefone_medico`  
    FOREIGN KEY (`telefone`)  
    REFERENCES `Trabalho_prático`.`TELEFONE` (`telefone`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION,  
  CONSTRAINT `fk2_hospital_medico`  
    FOREIGN KEY (`hospital_id`)  
    REFERENCES `Trabalho_prático`.`HOSPITAL` (`id_hospital`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION,  
  CONSTRAINT `fk3_cp_medico`  
    FOREIGN KEY (`codigo_postal`)  
    REFERENCES `Trabalho_prático`.`CODIGO_POSTAL` (`n_codigo_postal`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION,  
  CONSTRAINT `fk5_especialidade_medico`  
    FOREIGN KEY (`especialidade`)  
    REFERENCES `Trabalho_prático`.`ESPECIALIDADE` (`id_especialidade`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

Figura 38: Modelo Físico “MEDICO”

```
CREATE TABLE IF NOT EXISTS `Trabalho_prático`.`TRIAGEM` (  
  `id_triagem` INT NOT NULL,  
  `pulseira` VARCHAR(40) NOT NULL,  
  `sintomas` VARCHAR(200) NOT NULL,  
  PRIMARY KEY (`id_triagem`))  
ENGINE = InnoDB;
```

Figura 39: Modelo Físico “TRIAGEM”



```
CREATE TABLE IF NOT EXISTS `Trabalho_prático`.`UTENTE` (  
  `id_utente` INT NOT NULL,  
  `nome` VARCHAR(100) NOT NULL,  
  `codigo_postal` VARCHAR(20) NOT NULL,  
  `data_nascimento` DATE NOT NULL,  
  `telefone` INT(20) NOT NULL,  
  `triagem_id` INT NOT NULL,  
  PRIMARY KEY (`id_utente`),  
  INDEX `fk1_utente_telefone_idx` (`telefone` ASC) VISIBLE,  
  INDEX `fk2_cp_utente_idx` (`codigo_postal` ASC) VISIBLE,  
  INDEX `fk3_triagem_utente_idx` (`triagem_id` ASC) VISIBLE,  
  CONSTRAINT `fk1_utente_telefone`  
    FOREIGN KEY (`telefone`)  
      REFERENCES `Trabalho_prático`.`TELEFONE` (`telefone`)  
      ON DELETE NO ACTION  
      ON UPDATE NO ACTION,  
  CONSTRAINT `fk2_cp_utente`  
    FOREIGN KEY (`codigo_postal`)  
      REFERENCES `Trabalho_prático`.`CODIGO_POSTAL` (`n_codigo_postal`)  
      ON DELETE NO ACTION  
      ON UPDATE NO ACTION,  
  CONSTRAINT `fk3_triagem_utente`  
    FOREIGN KEY (`triagem_id`)  
      REFERENCES `Trabalho_prático`.`TRIAGEM` (`id_triagem`)  
      ON DELETE NO ACTION  
      ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

Figura 40: Modelo Físico “UTENTE”

```
CREATE TABLE IF NOT EXISTS `Trabalho_prático`.`DIAGNOSTICO` (  
  `id_diagnostico` INT NOT NULL,  
  `descricao` VARCHAR(500) NOT NULL,  
  PRIMARY KEY (`id_diagnostico`))  
ENGINE = InnoDB;
```

Figura 41: Modelo Físico “DIAGNOSTICO”



```
CREATE TABLE IF NOT EXISTS `Trabalho_prático`.`CONSULTA` (  
  `medico_id` INT NOT NULL,  
  `utente_id` INT NOT NULL,  
  `data_inicio` DATETIME NOT NULL,  
  `data_fim` DATETIME NOT NULL,  
  `diagnostico` INT NOT NULL,  
  PRIMARY KEY (`medico_id`, `utente_id`),  
  INDEX `fk2_utente_consulta_idx` (`utente_id` ASC) VISIBLE,  
  INDEX `fk3_diagnostico_consulta_idx` (`diagnostico` ASC) VISIBLE,  
  CONSTRAINT `fk1_medico_consulta`  
    FOREIGN KEY (`medico_id`)  
    REFERENCES `Trabalho_prático`.`MEDICO` (`id_medico`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION,  
  CONSTRAINT `fk2_utente_consulta`  
    FOREIGN KEY (`utente_id`)  
    REFERENCES `Trabalho_prático`.`UTENTE` (`id_utente`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION,  
  CONSTRAINT `fk3_diagnostico_consulta`  
    FOREIGN KEY (`diagnostico`)  
    REFERENCES `Trabalho_prático`.`DIAGNOSTICO` (`id_diagnostico`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

Figura 42: Modelo Físico “CONSULTA”

```
CREATE TABLE IF NOT EXISTS `Trabalho_prático`.`BLOCO` (  
  `id_bloco` INT NOT NULL,  
  `descricao` VARCHAR(500) NOT NULL,  
  `hospital_id` INT NOT NULL,  
  PRIMARY KEY (`id_bloco`),  
  INDEX `fk1_hospital_bloco_idx` (`hospital_id` ASC) VISIBLE,  
  CONSTRAINT `fk1_hospital_bloco`  
    FOREIGN KEY (`hospital_id`)  
    REFERENCES `Trabalho_prático`.`HOSPITAL` (`id_hospital`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

Figura 43: Modelo Físico “BLOCO”



```
CREATE TABLE IF NOT EXISTS `Trabalho_prático`.`CIRURGIA` (  
  `id_cirurgia` INT NOT NULL,  
  `data_realizacao` DATETIME NOT NULL,  
  `tempo_cirurgia` TIME NOT NULL,  
  `tipo_cirurgia` VARCHAR(100) NOT NULL,  
  `bloco_id` INT NOT NULL,  
  `utente_id` INT NOT NULL,  
  `urgencia` VARCHAR(45) NOT NULL,  
  `medico_id` INT NOT NULL,  
  PRIMARY KEY (`id_cirurgia`),  
  INDEX `fk1_utente_cirurgia_idx` (`utente_id` ASC) VISIBLE,  
  INDEX `fk2_bloco_cirurgia_idx` (`bloco_id` ASC) VISIBLE,  
  INDEX `fk3_medico_cirurgia_idx` (`medico_id` ASC) VISIBLE,  
  CONSTRAINT `fk1_utente_cirurgia`  
    FOREIGN KEY (`utente_id`)  
    REFERENCES `Trabalho_prático`.`UTENTE` (`id_utente`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION,  
  CONSTRAINT `fk2_bloco_cirurgia`  
    FOREIGN KEY (`bloco_id`)  
    REFERENCES `Trabalho_prático`.`BLOCO` (`id_bloco`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION,  
  CONSTRAINT `fk3_medico_cirurgia`  
    FOREIGN KEY (`medico_id`)  
    REFERENCES `Trabalho_prático`.`MEDICO` (`id_medico`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION)  
ENGINE = InnoDB;
```

Figura 44: Modelo Físico “CIRURGIA”

12.2 Anexo B: Povoamento da entidade “TELEFONE”

```
INSERT INTO Trabalho_prático.TELEFONE (telefone) VALUES
(911234567),
(921234567),
(931234567),
(961234567),
(912345678),
(922345678),
(932345678),
(962345678),
(912345680),
(932345680),
(952345680),
(962345680),
(972345680),
(912345681),
(932345681),
(952345681),
(911234568),
(921234568),
(931234568),
(961234568),
(911234569),
(921234569),
(931234569),
(961234569),
(213456789),
(223456789),
(239456789),
(253456789);
```

Figura 45: Povoamento da entidade “TELEFONE”

12.3 Anexo C: Povoamento da entidade “HOSPITAL”

```
INSERT INTO Trabalho_prático.HOSPITAL (id_hospital, nome, codigo_postal, telefone) VALUES
(1, 'Hospital de Santa Maria', '1000-001', 213456789),
(2, 'Hospital de São João', '4000-001', 223456789),
(3, 'Centro Hospitalar de Coimbra', '3000-001', 239456789),
(4, 'Hospital de Braga', '4700-001', 253456789);

select * from HOSPITAL;
```

Figura 46: Povoamento da entidade “HOSPITAL”

12.4 Anexo D: Povoamento da entidade “CODIGO_POSTAL”

```
INSERT INTO Trabalho_prático.CODIGO_POSTAL (n_codigo_postal, localidade) VALUES
('1000-001', 'Lisboa'),
('1000-002', 'Lisboa'),
('1000-003', 'Lisboa'),
('1000-004', 'Lisboa'),
('1000-005', 'Lisboa'),
('1000-006', 'Lisboa'),
('1000-007', 'Lisboa'),
('4000-001', 'Porto'),
('4000-002', 'Porto'),
('4000-003', 'Porto'),
('4000-004', 'Porto'),
('4000-005', 'Porto'),
('4000-006', 'Porto'),
('4000-007', 'Porto'),
('3000-001', 'Coimbra'),
('3000-002', 'Coimbra'),
('3000-003', 'Coimbra'),
('3000-004', 'Coimbra'),
('3000-005', 'Coimbra'),
('3000-006', 'Coimbra'),
('3000-007', 'Coimbra'),
('4700-001', 'Braga'),
('4700-002', 'Braga'),
('4700-003', 'Braga'),
('4700-004', 'Braga'),
('4700-005', 'Braga'),
('4700-006', 'Braga'),
('4700-008', 'Braga');
```

Figura 47: Povoamento da entidade “CODIGO_POSTAL”

12.5 Anexo E: Povoamento da entidade “TRIAGEM”

```
INSERT INTO Trabalho_prático.TRIAGEM (id_triagem, pulseira, sintomas) VALUES
(1, 'Verde', 'Dor de cabeça leve'), -- Nada
(2, 'Amarela', 'Dor persistente numa articulação'), -- Ortopedia
(3, 'Vermelha', 'Dor no peito'), -- Cardiologia
(4, 'Amarela', 'Dor de cabeça intensa e persistente'), -- neurologia
(5, 'Vermelha', 'Fraqueza súbita num lado do corpo'), -- Neurologia
(6, 'Vermelha', 'Dificuldade repentina para falar ou entender a fala'), -- Neurologia
(7, 'Amarela', 'Falta de ar'), -- Cardiologia
(8, 'Vermelha', 'Erupções cutâneas incomuns'), -- Dermatologia
(9, 'Amarela', 'Alergia ao Sol'), -- Dermatologia
(10, 'Verde', 'Dificuldade para movimentar uma parte do corpo ou caminhar'); -- Ortopedia
```

Figura 48: Povoamento da entidade “TRIAGEM”



12.6 Anexo F: Povoamento da entidade “UTENTE”

```
INSERT INTO Trabalho_prático.UTENTE (id_utente, nome, codigo_postal, data_nascimento, telefone, triagem_id) VALUES
(1, 'Carlos Mendes', '1000-003', '1980-02-15', 912345678, 1),
(2, 'Ana Rita', '4000-003', '1995-07-20', 922345678, 3),
(3, 'Joaquim Sousa', '3000-003', '1970-11-30', 932345678, 10),
(4, 'Maria João', '4700-003', '1985-03-25', 962345678, 2),
(5, 'João Silva', '1000-004', '1980-02-15', 912345680, 7),
(6, 'Maria Ferreira', '4000-004', '1980-02-15', 932345680, 5),
(7, 'Luís Oliveira', '3000-004', '1980-02-15', 952345680, 8),
(8, 'Ana Costa', '4700-004', '1980-02-15', 962345680, 10),
(9, 'Pedro Sousa', '1000-005', '1980-02-15', 972345680, 6),
(10, 'Sofia Martins', '4000-005', '1980-02-15', 912345681, 7),
(11, 'Tiago Rodrigues', '3000-005', '1980-02-15', 932345681, 9),
(12, 'Beatriz Cunha', '4700-005', '1003-09-16', 952345681, 8);
```

Figura 49: Povoamento da entidade “UTENTE”

12.7 Anexo G: Povoamento da entidade “DIAGNOSTICO”

```
INSERT INTO Trabalho_prático.DIAGNOSTICO (id_diagnostico, descricao) VALUES
(1, 'Gripe'),
(2, 'Osteoartrite'),
(3, 'Angina'),
(4, 'Cefalia crónica'),
(5, 'AVC'),
(6, 'Sangue nos pulmões'),
(7, 'Dermatite atópica'),
(8, 'Urticária solar'),
(9, 'Fémur fraturado');
```

Figura 50: Povoamento da entidade “DIAGNOSTICO”

12.8 Anexo H: Povoamento da entidade “MEDICO”

```
-----
INSERT INTO Trabalho_prático.MEDICO (id_medico, nome, data_nascimento, data_inicio_servico, especialidade, codigo_postal, telefone, hospital_id) VALUES
(1, 'João Silva', '1975-06-15', '2000-01-01', 1, '1000-002', 911234567, 1),
(2, 'Maria Fernandes', '1980-08-20', '2005-05-10', 2, '4000-002', 921234567, 2),
(3, 'Pedro Costa', '1979-12-01', '2002-03-15', 3, '3000-002', 931234567, 3),
(4, 'Ana Oliveira', '1985-04-10', '2010-11-01', 3, '4700-002', 961234567, 4),
(5, 'Manuel Santos', '1976-09-25', '2001-07-20', 2, '1000-006', 911234568, 1),
(6, 'Sofia Sousa', '1983-03-12', '2007-09-30', 1, '4000-006', 921234568, 2),
(7, 'Luís Pereira', '1974-11-18', '2003-10-05', 4, '3000-006', 931234568, 3),
(8, 'Carolina Martins', '1988-07-05', '2012-12-10', 3, '4700-006', 961234568, 4),
(9, 'Ricardo Ferreira', '1982-05-30', '2006-08-15', 5, '1000-007', 911234569, 1),
(10, 'Marta Santos', '1977-10-08', '2004-04-25', 5, '4000-007', 921234569, 2),
(11, 'Miguel Gonçalves', '1986-02-14', '2011-06-20', 5, '3000-007', 931234569, 3),
(12, 'Inês Almeida', '1973-08-22', '2000-12-05', 5, '4700-008', 961234569, 4);

select * from MEDICO;
```

Figura 51: Povoamento da entidade “MEDICO”

12.9 Anexo I: Povoamento da entidade “CONSULTA”

```
INSERT INTO Trabalho_prático.CONSULTA (medico_id, utente_id, data_inicio, data_fim, diagnóstico) VALUES
(9, 1, '2024-05-01 10:00:00', '2024-05-01 10:30:00', 1),
(10, 2, '2024-05-02 11:00:00', '2024-05-02 11:45:00', 3),
(11, 3, '2024-05-03 14:00:00', '2024-05-03 14:30:00', 9),
(12, 4, '2024-05-04 16:00:00', '2024-05-04 16:30:00', 2),
(9, 5, '2024-05-05 09:00:00', '2024-05-05 09:30:00', 6),
(10, 6, '2024-05-06 10:30:00', '2024-05-06 11:00:00', 5),
(11, 7, '2024-05-07 12:00:00', '2024-05-07 12:30:00', 7),
(12, 8, '2024-05-08 13:00:00', '2024-05-08 13:45:00', 9),
(9, 9, '2024-05-09 14:30:00', '2024-05-09 15:00:00', 5),
(10, 10, '2024-05-10 15:30:00', '2024-05-10 16:00:00', 6),
(11, 11, '2024-05-11 16:30:00', '2024-05-11 17:00:00', 7),
(12, 12, '2024-05-12 17:00:00', '2024-05-12 17:30:00', 12);
```

Figura 52: Povoamento da entidade “CONSULTA”

12.10 Anexo J: Povoamento da entidade “BLOCO”

```
INSERT INTO Trabalho_prático.BLOCO (id_bloco, descricao, hospital_id) VALUES
(1, 'Bloco Cirúrgico A', 1),
(2, 'Bloco Cirúrgico B', 2),
(3, 'Bloco Cirúrgico C', 3),
(4, 'Bloco Cirúrgico D', 4),
(5, 'Bloco Cirúrgico E', 1),
(6, 'Bloco Cirúrgico F', 2),
(7, 'Bloco Cirúrgico G', 3),
(8, 'Bloco Cirúrgico H', 4);
```

Figura 53: Povoamento da entidade “BLOCO”

12.11 Anexo K: Povoamento da entidade “CIRURGIA”

```
INSERT INTO Trabalho_prático.CIRURGIA (id_cirurgia, data_realizacao, tempo_cirurgia, tipo_cirurgia, bloco_id, utente_id, urgencia, medico_id) VALUES
(1, '2024-06-01 08:00:00', '02:00:00', 'Cardiologia', 2, 2, 'urgente', 6),
(2, '2024-06-02 09:00:00', '05:00:00', 'Ortopedia', 3, 3, 'urgente', 3),
(3, '2024-06-03 10:00:00', '07:00:00', 'Cardiologia', 1, 5, 'muito urgente', 1),
(4, '2024-06-04 11:00:00', '02:00:00', 'Neurologia', 6, 6, 'muito urgente', 2),
(5, '2024-06-05 12:00:00', '03:00:00', 'Dermatologia', 7, 7, 'pouco urgente', 7),
(6, '2024-06-06 13:00:00', '04:00:00', 'Ortopedia', 4, 8, 'urgente', 8),
(7, '2024-06-07 14:00:00', '06:00:00', 'Neurologia', 5, 9, 'muito urgente', 5),
(8, '2024-06-08 15:00:00', '02:30:00', 'Cardiologia', 2, 10, 'muito urgente', 6),
(9, '2024-06-10 17:00:00', '03:00:00', 'Dermatologia', 8, 12, 'pouco urgente', 4);

select * from CIRURGIA;
```

Figura 54: Povoamento da entidade “CIRURGIA”